

Prof. Dr. Armin Biere
Dr. Mathias Fleury

Freiburg, 28/03/2024

Final Exam

Computer Architecture

Name: _____

Matr. no.: _____

Number of pages: 17

Available time: 90 Minuten

Allowed resources: None

Please check if you have received **all** sheets and fill in your name and your matriculation number on **all** sheets (also on the additional sheets you have received). Sheets without this information will not be evaluated.

Question	Number of points	
	Possible	Aquired
1	16	
2	16	
3	16	
4	16	
5	16	
6	16	
Total	96	

Achieving **48** points is sufficient to pass the exam.

WITHDRAWING FROM AN EXAMINATION

In case of illness, you must supply proof of your illness by submitting a medical report to the Examinations Office. Please note that the medical examination must be done at the latest on the same day of the missed exam. In case of illness while writing the exam please contact the supervisory staff, inform them about your illness and immediately see your doctor. The medical certificate must be submitted latest 3 days after the medical examination. More information's: https://www.tf.uni-freiburg.de/studies/exams/withdrawing_exam.html

CHEATING/DISTURBING IN EXAMINATIONS

A student who disrupts the orderly proceedings of an examination will be excluded from the remainder of the exam by the respective examiners or invigilators. In such a case, the written exam of the student in question will be graded as 'nicht bestanden' (5.0, fail) on the grounds of cheating. In severe cases, the Board of Examiners will exclude the student from further examinations.

General information through this exam:

- Write your answers in Austrian, English, French, or German (alphabetical ordering). You can change between questions, but do not change within sentences.
- Whenever we ask for RISC-V code, we do not care about the exact instruction name. If you remember it, fine. If not, give it a descriptive acronym or name!
- Unless mentioned otherwise, we only consider 32-bit RISC-V.
- Unless specified otherwise, if we ask you to write programs, you do not need to save registers, nor remember which registers have to be saved, nor remember how many registers there are.
- Every exercise gives the same number of points.
- If asked to consider the time or number of cycles to execute a program in a pipelined architecture, you do not need to consider the warm-up / close-down phase (i.e., you can consider that one instruction requires 1 cycle).
- This page, page 3 and page 10 have additional space for all solutions. If needed make sure to indicate **both**
 - on the additional page, which exercise your solution refers to;
 - in the corresponding exercise, that you used an additional page *with the page number*.

Failing to do so ends up in an exercise that is only partially corrected.

Name: _____

Matriculation number: _____

3

Additional pages for all exercises

Exercise 1 (2+5+5+4 points)

In this exercise, we are going to work on structures and how they are laid out in memory.

We distinguish between *aligned* and *unaligned* memory reads. A memory read is *aligned* when memory is read starting at a word boundary and unaligned if not. We consider that words have 4 bytes. Let's consider the following memory representation:

```
| - - - - | - - - - | - - - - |
  0         4         8
```

If you start reading 4 bytes at a multiple of 4 (0, 4, 8, ...), the reading is aligned. Otherwise it is not aligned. Reading unaligned memory is *slower* than reading aligned memory.

We consider the following C structure called **Aligned**

```
struct Aligned {
    bool a:1; /*1 bits*/
    int b;    /*32 bits*/
};
```

- a) Give the representation of **Aligned** in memory. Make sure that every member is aligned and that you use little-endian: if you use 3 bytes in a 4-byte region, you should use the 3 right-most ones and leave the first unused. The value of the unused bits is random.

```
| - - - - | - - - - | - - - - |
  0         4         8
```

- b) Complete the code below that multiplies two **Aligned** element-wise ($a*a$ and $b*b$)

```
lw x, __a0 ; a0 is the first Aligned to multiply
lw y, __a1 ; a1 is the other
mul x, x, y
sw x, __a2 ; a2 is the destination Aligned
lw x, __a0
lw y, __a1
----- ; do something on x
----- ; do something on y
and x, y, y
sw x, __a2
```

Name: _____

Matriculation number: _____

5

- c) Our RISC-V processor has a RISC-V instruction `mulps` that can multiply 2 blocks of 4 consecutive `ints` together. In order to use them, they must be stored one after the other in memory, as on one array of `ints` and as one array of `bools`.

Write some pseudo-but-realistic RISC-V for the implementation using `mulps` (only for the multiplication on integers).

- d) For efficiency, we want to keep the `Aligned` structure, but we want to use `mulps` and load our `bool` as an integer. What changes in the assumptions on the memory? How can we use `mulps` for the Boolean?

Exercise 2 (2+8+3+3 points)

In this exercise, we study the branch prediction and we consider a RISC-V CPU with a branch predictor based on an old AMD processor (Athlon and Phenom CPUs).

Our CPU has a pipeline of *three* stages, called A, B, and C. You may assume that in the programs we are considering there are no data dependencies between instructions. **Branching instructions are resolved at the end of the last pipeline stage C.** In the mean time the CPU continues executing instructions as predicted by a two-bit branch predictor.

Our CPU has a cache for instructions (L1i) with a capacity of only *three* instructions. The L1i cache contains the current instruction (given by the PC) and two surrounding instructions. More precisely, it contains the instructions at $\lfloor \frac{PC}{3} \rfloor$, $\lfloor \frac{PC}{3} \rfloor + 1$, and $\lfloor \frac{PC}{3} \rfloor + 2$. For example, when executing any of the instructions 3, 4, or 5, the L1i cache contains the instructions 3, 4, and 5. **All three entries are populated at once at each step.**

The information to remember the status of the **two-bit** branch predictor is included in a separate register file. To reduce its size, it contains only 3 entries with number 0, 1, and 2. The predictor looks up at the last digit of the current address to get the current status (like 20211_2 gives 1).

a) Explain by drawing an automaton (with 4 states) how the **two-bit** branch predictor works.

b) We consider the following program with the given instruction addresses:

010 ₃ :	beq y, 0, 021 ₃
011 ₃ :	addi x, x, 4
012 ₃ :	ld y, a0(x)
020 ₃ :	bgei x, 120, 010 ₃
021 ₃ :	nop
022 ₃ :	nop

Fill out what is happening to the L1i cache when executing the program on the next page:

- fill in the content of the instruction cache (including addresses) during the execution and mark the current instruction with an arrow, as shown for the first step in the execution
- the branch prediction shows the state *before* the execution of the current pipeline stage
- stop as soon you finished **bgei** and updated the branch prediction
- assume that branches **beq** at address 010₃ is never taken, while the **bge** at address 020₃ is always taken
- the light gray lines have no semantics and can be ignored
- the number of empty steps templates given might be longer than necessary
- only add content in the branch predictor when it changes: as there is no branch with address 2, the status will never change and you do not have to write anything.

Name: _____

Matriculation number: _____

7

	branch predictor
0	always take
1	always take
2	always take

→	L1i instructions	branch predictor
	010 ₃	0
	011 ₃	1
	012 ₃	2

A B C

	L1i instructions	branch predictor
	--0 ₃	0
	--1 ₃	1
	--2 ₃	2

A B C

	L1i instructions	branch predictor
	--0 ₃	0
	--1 ₃	1
	--2 ₃	2

A B C

	L1i instructions	branch predictor
	--0 ₃	0
	--1 ₃	1
	--2 ₃	2

A B C

	L1i instructions	branch predictor
	--0 ₃	0
	--1 ₃	1
	--2 ₃	2

A B C

	L1i instructions	branch predictor
	--0 ₃	0
	--1 ₃	1
	--2 ₃	2

A B C

	L1i instructions	branch predictor
	--0 ₃	0
	--1 ₃	1
	--2 ₃	2

A B C

	L1i instructions	branch predictor
	--0 ₃	0
	--1 ₃	1
	--2 ₃	2

A B C

L1i instructions	branch predictor
--0 ₃	0
--1 ₃	1
--2 ₃	2

A B C

L1i instructions	branch predictor
--0 ₃	0
--1 ₃	1
--2 ₃	2

A B C

L1i instructions	branch predictor
--0 ₃	0
--1 ₃	1
--2 ₃	2

A B C

L1i instructions	branch predictor
--0 ₃	0
--1 ₃	1
--2 ₃	2

A B C

L1i instructions	branch predictor
--0 ₃	0
--1 ₃	1
--2 ₃	2

A B C

L1i instructions	branch predictor
--0 ₃	0
--1 ₃	1
--2 ₃	2

A B C

L1i instructions	branch predictor
--0 ₃	0
--1 ₃	1
--2 ₃	2

A B C

L1i instructions	branch predictor
--0 ₃	0
--1 ₃	1
--2 ₃	2

A B C

Name: _____

Matriculation number: _____

9

- c) What happens if the branch predictor starts in **never taken**? (Text, do not repeat the graph!)
- d) If you use a single digit to index the addresses for the branch predictor, is the last one like here the best choice? Why?

Additional pages for all exercises

Exercise 3 (3+5+2+6 points)

For this exercise, we will consider a two-bit divider (one sign bit and one digit) and we will verify it. We consider numbers similar to floating point: the sign bit tells you whether the number is negative or not.

We consider a divider such that: $\alpha = \begin{cases} \beta \times \text{result} & \text{if } \beta \neq 0 \\ \alpha & \text{otherwise} \end{cases}$.

The input are $\alpha = [s_1 a]_2$ and $\beta = [s_2 b]_2$. And the result is $\gamma = \frac{a}{b} = [sr]_2$ and an overflow bit o .

- a) Write one correctness equations over $\mathbb{Z}$ without case distinction. When dividing by zero, the result is $\gamma = \alpha$, but the overflow bit is set.

Hint: to make the calculations easier, keep all the $(-1)^x$ and do not transform them to polynomials.

- b) We consider a circuit calculating $\alpha \times \beta$ by returning the following three bits (sign, result, and overflow) from the inputs:

$$\begin{aligned} s &= (s_1 \oplus s_2) \\ r &= (a \wedge b) \vee (a \wedge \neg b) = a \\ o &= \neg b \end{aligned}$$

Translate all the equations as polynomials.

- c) Explain why and how you transform x^n in a version without exponent.
- d) Explain how you would prove the correctness of the multiplier w.r.t to the specification. Do the first three steps of the calculation.

Hint: the full calculation takes 5 steps, but we only ask you for the first three to make correction easier.

Your solution to exercise 3:

Additional pages for exercise 3 – page 3

Exercise 4 (2+8+6 points)

We use the usual five pipeline stages – IF, ID, EX, MEM, and WB – and the optimization with determining branch conditions for branches with a separate comparator/adder during the ID phase.

Assume that there is no branch prediction in this exercise, so the CPU waits for the branch destination to be found before executing the next instruction.

Throughout this exercise we further assume that the `beq` branch is **not taken**.

- a) Add arrows to represent all read after write dependencies between registers:

```
    lbu      a1, 4(a0)
    addi     a2, a1, 1024
    li       a1, 300
    beq      a2, x0, .LBB0_2 // not taken
    lb       a1, 1(a0)
    slri     a1, a1, 8
    lb       a2, 0(a0)
    and      a1, a0, a2
.LBB0_2:
    addi     a0, a5, zero
```

- b) Indicate in the same program below what kind of forwarding happens and from which pipeline stage (IF, ID, EX, MEM, WB) to which pipeline stage this happens:

```

        lbu      a1, 4(a0)

        addi     a2, a1, 1024

        li       a1, 300

        beq      a2, x0, .LBB0_2 // not taken

        lb       a1, 1(a0)

        slri     a1, a1, 8

        lb       a2, 0(a0)

        and      a1, a0, a2

.LBB0_2:
        addi     a0, a5, zero

```

- c) Indicate in the same program where stalls happens and what kind of hazard it is:

```

        lbu      a1, 4(a0)

        addi     a2, a1, 1024

        li       a1, 300

        beq      a2, x0, .LBB0_2 // not taken

        lb       a1, 1(a0)

        slri     a1, a1, 8

        lb       a2, 0(a0)

        and      a1, a0, a2

.LBB0_2:
        addi     a0, a5, zero

```

Exercise 5 (2+2+3+2+2+5 points)

We consider a new kind of CPU tiles (rumored to be called *royal core* by Intel). Each tile is composed of *two* RISC-V (energy efficient) cores and *one* Intel-like (performance) core (with out-of-order features). For the sake of the exercise, the instruction stream is RISC-V with all the SIMD extension that cannot be executed on the RISC-V cores. We consider only the RISC-V version from Chapter 4 of the lecture (so SIMD, no out-of-order, ...).

Each tiles decides whether to use the two RISC-V cores or the Intel part.

Feature	RISC-V	Intel
AVX-instruction	no	yes
Reservation stations	no	yes (4)
Frequency	2 GHz	2 GHz
Pipelining stages	5	20
Out-of-order	no	yes
Power	100W	300W

We consider programs that run the following distribution:

CPU	ALU/Logic	Jump/Branch	Load	Store
Average CPI for RISC-V	1	2	2	1
Average CPI for Intel	.25	1	.5	1
Percentage of instructions (%)	40	20	20	20

- Due to space constraint on the CPU, you can add either one advanced branch predictor or two simpler ones per tile. For which core should you add it? (Justify, around 2 sentences)
- One program is 3 times faster on Intel than on RISC-V for mathematical computations without jumps, while the frequency is only twice as fast. How is this possible?
- Give 3 reasons to scheduling on either core type and mention whether the tile can know just from the current instruction stream.
- Compared to previous generation, Intel did not manage to add support for SMT (hyper-threading). What parts of the CPU are missing compared to a usual Intel core with SMT?
- Explain why the load instruction `ld` is less critical on the Intel part than on the RISC-V part even when both are using the same cache and the same RAM.
- A program needs 100 billion instruction (Ginstructions) on the Intel part (fewer hazards), but 200 Ginstructions on RISC-V. Calculate the CPI and the amount of time required for both cores? Is this worth using Intel over RISC-V?

Your solution to exercise 5:

Additional pages for exercise 5 – page 3

Exercise 6 (2+2+2+2+2+4+2 points)

- a) Explain the difference between RISC and CISC.
- b) Explain cache associativity.
- c) Explain the advantage and disadvantage to increase cache associativity.
- d) Who is responsible for scheduling programs on a CPU core?
- e) Why is the **perf** tool useful? What does it do?
- f) List 4 CPU features not in the RISC-V version from Chapter 4 that make the runtime not predictable even when executing the program several times.
- g) Intel and TMC are building CPUs based on smaller and smaller transistors. Why? Why not just increase the frequency?